
Using Machine Learning to Identify and Classify Brain Tumors: Techniques and Analysis of the Best Models

Edoardo Simonetti Spallotta¹

Abstract

This report aims at providing a comprehensive overview of the best techniques and models that are applicable when dealing with a classification problem: given an MRI (Magnetic Resonance Imaging) file, the objective is to correctly classify whether the subject presents signs of a brain tumor or not, and which kind of tumor, if one is present. The ultimate goal was not to imply that we could substitute doctors; rather, it was to provide an alternative tool that medical staff could use to obtain a first analysis to support their work. This could provide guidance when dealing with ambiguous MRI scans and decrease human error in diagnosing brain tumors.

1. Overview of the problem

After identifying a set of candidate datasets, the most promising one was the [Brain Tumor Classification \(MRI\)](#) dataset (Sartaj Bhuvaji, Ankita Kadam, Prajakta Bhumkar, Sameer Dedge, Swati Kanchan), which is valid since it contains four different classes of tumors that were identified as suitable for the *multi-class classification task* I deemed relevant:

- Glioma Tumor
- Meningioma Tumor
- Pituitary Tumor
- No Tumor

Each class is sufficiently equally dense, with the *No Tumor* class being slightly under-represented, an aspect that was taken into consideration.

¹MSc in Engineering in Computer Science and Artificial Intelligence, Sapienza University of Rome, Italy. Correspondence to: Edoardo Simonetti Spallotta <simonettispallotta.2077561@studenti.uniroma1.it>.

1.1. Dataset Properties

The dataset was already divided into Training and Testing splits, but there were issues which arose during the testing phase of the models. After some careful inspection, it was clear that the problem was not inherently due to an issue with the logic behind how the different models were trained; rather, it stemmed from a splitting issue in the dataset. For this exact reason, the dataset needed some polishing, resulting in a new, shuffled split I curated. We will take a closer look at that aspect later in this paper.

The choice of this particular dataset was also influenced by the degree of entropy across the dataset: instead of rotating each image to be consistent with the sagittal plane, I chose to keep the images slightly rotated and not perfectly straight, as to increase the generalization power. As a result, the best performing models are able to analyze reliably any MRI scan regardless of the orientation of said scan. No color normalization was performed for this same reason, choosing to keep varying intensities of greyscale.

The dataset only contains .jpg file formats, for a total of 3264 images.

2. Methodology

In this section, we will take a closer look at how the best model was picked, covering every aspect of the pipeline, from data preprocessing to evaluating with rigorous methods the best performing model.

2.1. Data Preprocessing

As briefly mentioned in the previous section, the dataset was already divided into Training and Testing splits. I initially trusted the curators and used those splits, only to quickly find out at the *testing* phase some inconsistencies across the two splits. I reliably measured a large drop in *accuracy* between the validation stage conducted post training, and the testing stage, conducted after choosing the best performing model.

This road-bump led me to disregard the original splits and instead merge the dataset together to form a randomized one, to then split it again into:

- 70% of the original dataset for Training
- 15% for Validation, and
- 15% for Testing

This operation alone effectively fixed the imbalance I measured at the earliest stage of the development, confirming my doubts.

After merging the dataset, the standard `train_test_split()` function from `sklearn` was used to split the dataset as intended:

```
X_train_paths , X_temp_paths ,
y_train , y_temp = train_test_split(
image_paths ,
labels ,
test_size=0.3 ,
random_state=42 ,
stratify=labels )
```

```
# split temp into 15/15
```

```
X_val_paths , X_test_paths ,
y_val , y_test = train_test_split(
X_temp_paths , y_temp ,
test_size=0.5 ,
random_state=42 ,
stratify=y_temp )
```

The partition is done using a stratified split to preserve the original class distribution. All preprocessing steps (standardization and PCA projection where applicable) are fitted exclusively on the training split, ensuring that no information from the validation or test sets leaks into the training process.

After performing this crucial step, we can move to the next problem: *How do we represent an image via vectors to use the classical Machine Learning models we want?* I considered using Histogram of Oriented Graphs (HOGs) to tackle this problem, but due to that technique being somewhat dated I opted for using Convolutional Neural Networks (CNNs)

2.2. Using CNN to understand images

2.2.1. BRIEF OVERVIEW OF CNNs

Convolutional Neural Networks are a class of *deep learning models* designed to automatically and adaptively learn spatial patterns in data. They excel at image-related tasks thanks to the usage of **convolutional layers**, which apply small filters across an input to detect features such as edges, textures and shapes. As the network goes deeper, the layers learn increasingly abstract patterns, enabling the model to

recognize complex structures like objects or faces (or, in this case, tumors).

A typical CNN architecture includes:

1. Convolutional layers for feature extraction
2. Pooling layers to reduce spatial dimensions and improve efficiency
3. Activation functions (typically, ReLu) to introduce non-linearity
4. Fully connected layers for final classification and prediction

2.2.2. CHOOSING THE IDEAL CNN FOR THIS TASK

I decided not to fine-tune a model since the ultimate goal is fairly straight-forward, and instead use a frozen, pretrained model. The first iteration used the VGG16 model, which despite being a good candidate, has proven to fall short on creating the ideal embeddings. The next choice, which produced a much higher-quality output, was EfficientNetB0.

VGG16 is a deep, but relatively uniform, CNN with about 138M parameters, relying on stacked 3x3 convolutions and simple architectural design. While influential and essentially good, it is computationally heavy and does not use modern efficiency techniques. In contrast, EfficientNetB0 is a much smaller, much more optimized, model that uses *compound scaling, depthwise separable convolutions* and improved building blocks to achieve better accuracy with far fewer parameters ($\approx 5M$). The takeaway is that EfficientNetB0 tends to learn more effectively, generalize better and train faster.

Crucially, EfficientNetB0 produces more compact and more informative embeddings than VGG16, and lower-dimensional.

2.2.3. LETTING THE CNN UNDERSTAND

After choosing *EfficientNetB0* for the above reasons, the dataset is ready to be ingested into the CNN. Before doing this, we need to first preprocess the images: I picked 224×224 for the images dimension, proving to be a good middle ground between accuracy and computational load. The images are loaded and preprocessed according to the `efficientnet` preprocessing style via the `preprocess_input()` from the `tensorflow` library.

After this preliminary step, we can invoke the `predict()` function of the model on the processed data, extracting the **feature vectors**.

3. Training

The training section is where everything comes together, and we can finally get a first feedback on our work. We must first standardize the features to better prepare data for the classical Machine Learning models, and this is performed leveraging the `StandardScaler` module of `sklearn`.

Again, from `sklearn`, we get the standard models:

- Gaussian Naive Bayes
- Logistic Regression
- SVM (with RBF kernel)
- Decision Tree
- Random Forest

and, after running the global training step, we get the following results (reference Figure 1 for details):

Table 1. Validation accuracy of classical ML models.

Model	Accuracy
Naive Bayes	0.7553
Logistic Regression	0.8924
SVM (RBF)	0.9283
Decision Tree	0.7089
Random Forest	0.8819

By examining Figure 1, it appears clear that the **SVM with RBF kernel** is the best-performing model across all metrics. In particular, it achieves the highest overall validation accuracy (0.9283) and consistently strong per-class scores.

This suggests that the RBF (Radial Basis Function, Gaussian Kernel) kernel is able to capture well non-linear class boundaries in the embedding space far better than the linear methods, and more robustly than tree-based methods. This superior performance can be attributed to:

1. the embeddings being compact, smooth and well-separated (thanks to the CNN choice)
2. the RBF kernel’s ability to construct flexible decision surfaces
3. SVM’s margin-maximization, which reduces overfitting compared to Decision Trees and Random Forests

In conclusion, the SVM is the model which best exploits the geometry of the learned feature space.

Naive Bayes				
Class	Precision	Recall	F1-score	Support
glioma_tumor	0.77	0.74	0.75	139
meningioma_tumor	0.74	0.50	0.60	140
no_tumor	0.71	0.97	0.82	60
pituitary_tumor	0.78	0.94	0.85	135
Accuracy				0.76 (N=474)
Logistic Regression				
Class	Precision	Recall	F1-score	Support
glioma_tumor	0.88	0.87	0.88	139
meningioma_tumor	0.82	0.83	0.82	140
no_tumor	0.96	0.92	0.94	60
pituitary_tumor	0.95	0.97	0.96	135
Accuracy				0.89 (N=474)
SVM (RBF)				
Class	Precision	Recall	F1-score	Support
glioma_tumor	0.97	0.84	0.90	139
meningioma_tumor	0.84	0.94	0.89	140
no_tumor	1.00	0.98	0.99	60
pituitary_tumor	0.97	0.98	0.97	135
Accuracy				0.93 (N=474)
Decision Tree				
Class	Precision	Recall	F1-score	Support
glioma_tumor	0.75	0.71	0.73	139
meningioma_tumor	0.61	0.60	0.60	140
no_tumor	0.72	0.70	0.71	60
pituitary_tumor	0.76	0.83	0.79	135
Accuracy				0.71 (N=474)
Random Forest				
Class	Precision	Recall	F1-score	Support
glioma_tumor	0.91	0.82	0.86	139
meningioma_tumor	0.79	0.86	0.82	140
no_tumor	0.98	0.87	0.92	60
pituitary_tumor	0.92	0.97	0.95	135
Accuracy				0.88 (N=474)

Figure 1. Per-class validation performance of classical ML models on the brain tumor classification task.

Examining Other Models The second-best performer, which is often used as a benchmark, is the **Logistic Regression**. It yielded an accuracy of 0.8924, which is surprisingly good despite being linear. The other metrics are consistently high, too. This suggests that the embedding space has *good linear separability*, but still benefits from the non-linear boundary modeling of SVM.

The **Random Forest** is a close third, with an accuracy of 0.8819. Although tree ensembles can model non-linearities, it could be the case that our representation is too dense and high-level learned to be applicable for a tree method, favoring instead *hyperplane-based methods* like SVM and Logistic Regression.

Naive Bayes (NB) is a poor performer here, displaying an accuracy of 0.7553. This is due to the assumption behind a NB classifier: **feature independence**. Deep embeddings strongly violate this assumption. As a result, the *Recall* for

meningioma_tumor drops to 0.50, and *Precision and Recall* imbalance is substantial across classes. A NB classifier simply cannot model the complex correlations present in our representation space.

The worst overall performer in the validation step is the **Decision Tree**. As expected, it overfits easily, does not exploit the smoothness or structure of the embedding manifold and struggles with multi-class boundary complexity. This results in mediocre per-class performance and the lowest overall accuracy at 0.7089.

3.1. Hyperparameter Tuning with Cross-Validation

As we can see from the results on the validation set, the best performing model is **SVM with the RBF kernel**. We can therefore proceed with tuning its hyper-parameters leveraging the *Grid Search Cross-Validation*, with the goal of finding the best values of C and γ . C is the penalty for misclassifications, hence a large C implies a strict margin, but risks overfitting. A small C instead implies a soft margin, allowing for some misclassifications and better generalization. Finding the best γ means tuning the parameter of the RBF Kernel that controls how far the influence of each training point reaches:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (1)$$

The parameter Gamma (γ) is directly proportional to the rate of decay of the RBF kernel. In other words, a high gamma implies a small radius of influence, leading to a model prone to overfitting, while a low gamma implies a large radius, therefore better generalization.

After concatenating the Validation and Train splits to gain more data points, we define quickly the hyper-parameters to try:

Table 2. Hyperparameter search space for SVM (RBF).

Parameter	Value
C	0.1
C'	1
C''	10
C'''	100
γ	scale
γ'	auto
γ''	0.01
γ'''	0.001

and, by trying every combination of the above hyper-parameters, we have **16 models**, each evaluated by 5-fold CV, which ultimately gives us **80 SVM trainings**. The best model is yet again chosen depending on the best accuracy. We will keep *Logistic Regression* as a baseline, with default hyper parameters.

After identifying the best hyper parameters for SVM, we

finally perform a classification run on the test split:

SVM (RBF) – Test Set Results

Class	Precision	Recall	F1-score	Support
glioma	0.90	0.88	0.89	139
meningioma	0.89	0.90	0.90	141
no_tumor	0.95	0.97	0.96	75
pituitary	0.98	0.99	0.98	135
Accuracy				0.93 (N=490)

Logistic (Softmax) Regression – Test Set Results

Class	Precision	Recall	F1-score	Support
glioma	0.88	0.87	0.88	139
meningioma	0.87	0.89	0.88	141
no_tumor	0.96	0.93	0.95	75
pituitary	0.95	0.96	0.96	135
Accuracy				0.91 (N=490)

Observations While the SVM clearly is the best performer, it is interesting to notice how Logistic Regression still performs quite well, especially considering it received no hyperparameter tuning and it being a linear model. This can be attributed to, again, the CNN producing quasi-linearly separable features; the embedding space forming well-separated clusters; decision boundaries between tumor classes that are close to being linear.

These characteristics are to be expected using **EfficientNet embeddings**, which are, as noted earlier:

- compact
- discriminative
- smooth and structured
- designed to separate semantic categories cleanly

We can therefore conclude that there is enough empirical evidence to understand the representation space is very well-organized.

4. Analysis and Evaluation

We now turn to the evaluation phase, offering a comprehensive quantitative and qualitative analysis of the SVM model’s performance. In particular, we will analyze:

- The Confusion Matrix
- The ROC curves
- Learning Curves (across all models, for completeness)
- Decision Boundaries
- Feature Importance

4.1. Confusion Matrix

By computing the confusion matrix we can appreciate a pronounced diagonal pattern, further confirming the quality of the model.

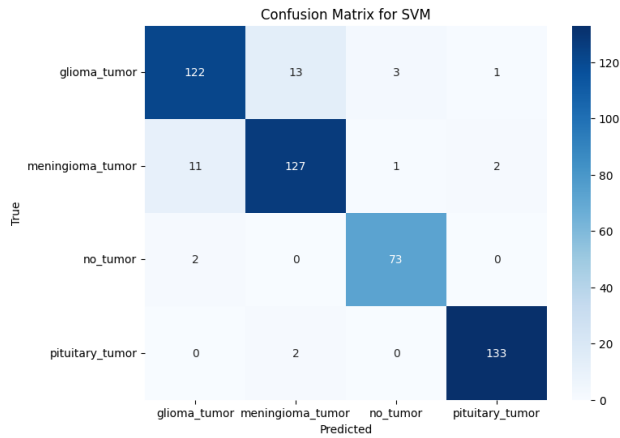


Figure 2. Confusion Matrix: SVM(RBF Kernel)

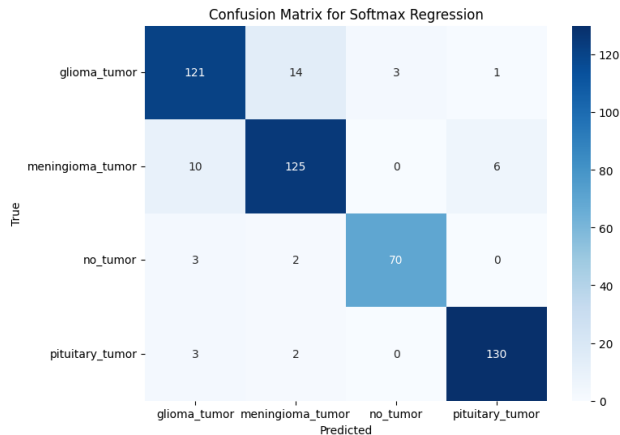


Figure 3. Confusion Matrix: Softmax Regression

We can see in Figure 2 that the SVM classifier achieves consistently strong performances across all classes, with predictions concentrated along the diagonal and presenting a small number of off-diagonal errors. Namely, the *Glioma Tumor* and the *Meningioma Tumor* are the classes that the model struggles with the most, and this is biologically plausible since these tumors share similar visual characteristics (location, shape) and they are known to be the hardest pair to distinguish.

This further confirms the good separation between the **tumor** and **no_tumor** classes. Furthermore, the cleanest sep-

aration appears to be among pituitary tumors, where 133 out of 135 pituitary cases are correctly classified. We can conclude that the pituitary tumors form a **distinct, well-clustered region** in the embedding space.

As a takeaway, the confusion matrix exhibits strong diagonal dominance, including high classification accuracy across all classes. The most frequent confusion occurs between glioma and meningioma tumors, known in the literature as being harder to distinct. Overall, the matrix confirms that the SVM model produces highly reliable, well-structured predictions, consistent with the quantitative F1 scores.

Briefly glancing over the LR confusion matrix, we can appreciate how they both share similar characteristics, with strong diagonal dominance and a limited number of misclassifications. As with the SVM, most errors occur between glioma and meningioma tumors. Compared to the SVM, LR produces slightly more off-diagonal errors, particularly among the tumor classes, a behavior that is consistent with its lower F1 scores.

Overall, the matrix confirms that Logistic Regression provides a strong linear baseline, but the SVM is able to capture additional nonlinear structure and resolves ambiguities more efficiently.

4.2. ROC Curves and AUC

We now examine the Receiver Operating Characteristic (ROC) curves and their corresponding Area Under the Curve (AUC) values. The ROC curve provides a threshold-independent view of how well a classifier separates positive from negative samples by plotting the True Positive Rate against the False Positive Rate.

Since ROC analysis is inherently binary, we extend it to the multi-class setting using the **One-vs-Rest (OvR)** strategy. For each class c , we construct a binary discrimination problem by treating:

- samples belonging to class c as the **positive** class, and
- samples from all remaining classes as the **negative** class.

This yields four independent ROC curves:

- *glioma_tumor vs. rest*
- *meningioma_tumor vs. rest*
- *no_tumor vs. rest*
- *pituitary_tumor vs. rest*

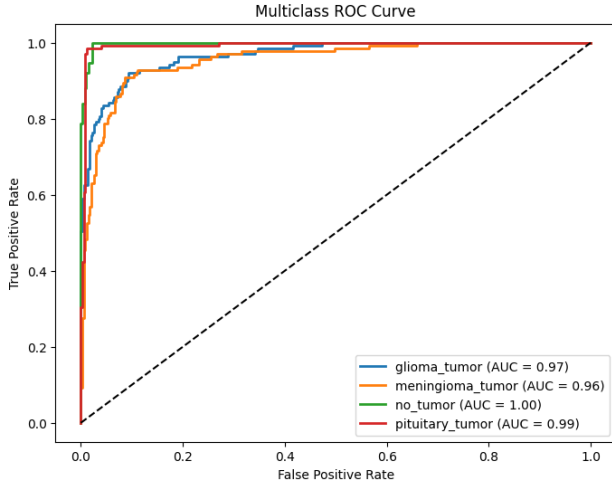


Figure 4. One-vs-Rest ROC curves and corresponding AUC values for each class.

As shown in Figure 4, all classes achieve high separability, with ROC curves concentrated in the upper-left region. In particular, the `no_tumor` class reaches an AUC of 1.00, indicating perfect ranking ability. These results corroborate our earlier findings: EfficientNet embeddings produce a highly discriminative feature space, enabling the SVM classifier to construct reliable decision boundaries even in a multi-class setting.

4.3. Learning Curves

We shift our focus now on examining learning curves, which illustrate how performance evolves as a function of the training set size.

Learning curves provide a valuable insight into the model’s generalization properties, revealing whether the classifier suffers from high variance or high bias, whether additional data could improve performance, and whether the training process is stable.

We can also get a better understanding of how efficiently the model learns the underlying decision boundaries and whether its capacity matches the complexity of the task.

Figure 5 reveals a model with **low bias** and **moderate variance**, with stable and improving generalizations as more training data is provided. The training accuracy remains consistently high (around 0.97-0.98) across all training sizes, indicating the SVM is sufficiently expressive to fit the training data very well with small sample subsets.

In contrast, the validation accuracy starts significantly lower for small training sizes (around 0.65 at 200 samples), reflecting underfitting due to insufficient data to refine reliable decision boundaries. However, the validation performance

increases steadily with the number of training samples, and stabilizes at around 0.88. This indicates that the model benefits greatly from more data and does not saturate early.

The consistent, albeit narrowing, gap between the two curves suggests some **mild overfitting**, which is somewhat typical for high-capacity nonlinear models, especially those trained on deep embeddings.

Given the upward trend in validation accuracy and the decreasing variance, the generalization performance continues to improve with additional data.

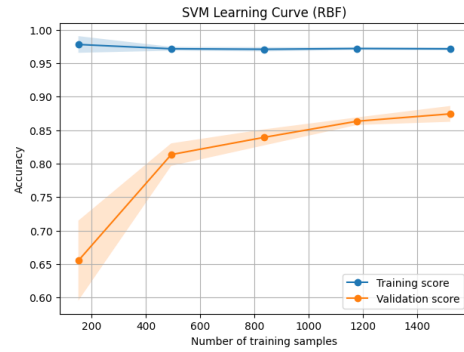


Figure 5. Learning Curves (SVM)

On the other hand, the performance of the Logistic Regression in Figure 6 shows a markedly different behavior. The training accuracy stays almost constant at 1.00 which reflects the fact that the Logistic Regression fits the EfficientNet feature space extremely well, from the very beginning. However, unlike the SVM, the validation accuracy increases more gradually, sitting at around 0.74 and stabilizing near 0.87 as more data becomes available.

The relatively small gap between the training and validation curves indicate **lower model variance** and a smoother generalization profile.

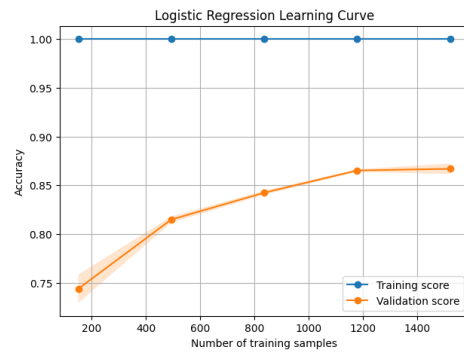


Figure 6. Learning Curves (LR)

Logistic Regression converges quickly because its linear decision boundary has lower capacity than the nonlinear SVM, meaning it does not overfit as easily but also cannot fully capture subtler class distinctions. We can also notice how more data does not translate to better accuracy, a typical behavior of a linear classifier operating in a discriminative embedding space.

Overall, the learning curve confirms that Logistic Regression serves as a strong linear baseline, benefiting from the structure of the feature space but ultimately limited by its linearity.

The reader can compute the rest of the learning curves in the associated notebook, omitted here for brevity, since they do not contribute significantly to the scope of this paper.

4.4. Decision Boundaries

By definition, a decision boundary is the set of points in the feature space where two classes have equal probability (or equal decision score). While normally intuitive, in our high-dimensional space (produced by EfficientNet), the boundaries are not interpretable and impossible to visualize directly. For this reason, I chose to project them in a 2D space, leveraging **Principal Component Analysis (PCA)** for dimensionality reduction with the ultimate goal of producing approximately linear decision boundaries. This process involves training an additional Logistic Regression model only on the 2D projections, with the tradeoff that the output is *not* how SVM views the decision bounds, rather an approximation of how separable the dataset is in the principal subspace.

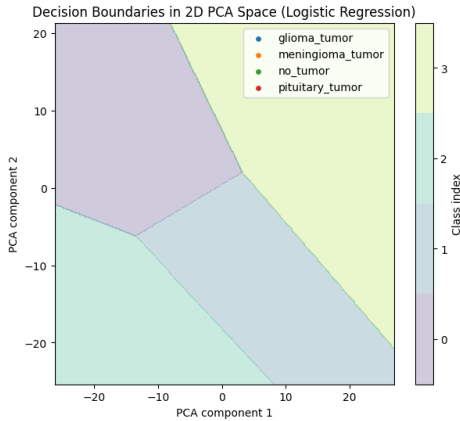


Figure 7. Decision Boundaries (PCA 2D)

The resulting plot (Figure 7) shows how the four classes produce distinct clusters, indicating that even after an aggressive dimensionality reduction, the embeddings remain well separated. This confirms the high discriminative qual-

ity of the learned representations, consistent with what we observed in our previous quantitative evaluations.

Naturally, the boundary between the *meningioma* and *glioma* tumor clusters are narrow, in line with the slightly elevated mutual misclassification rates highlighted in the confusion matrix between those same classes.

In conclusion, while it is important to note that this is a simplified 2D approximation, with the actual space likely being more curved and complex, this provides an intuitive and interpretable summary of the overall structure of the feature space and relationships between classes.

4.5. Feature Importance

Although the SVM with RBF kernel is the best-performing model in our experiments, *it does not provide a meaningful notion of feature importance*. This is because the RBF kernel operates in an implicit high-dimensional feature space, where decisions are made through pairwise similarities between support vectors rather than through explicit weights on individual input dimensions. As a result, no direct attribution can be assigned to each EfficientNet feature: to still obtain some interpretability over the embedding space, we compute feature importance using the Logistic Regression baseline. Unlike the SVM, Logistic Regression learns an explicit weight vector whose coefficients indicate how strongly each feature contributes to the class decision. This choice maintains consistency with our baseline-versus-SVM comparison while avoiding the introduction of additional models solely for interpretability.

Figure 8 reports the top 20 feature importances for the Logistic Regression baseline, measured as the ℓ_2 norm of each coefficient vector across classes:

$$\text{importance}(j) = \|\text{coef}_{:,j}\|_2 \quad (2)$$

The most influential embedding dimensions (e.g. features *f1249*, *f951*, *f1045*) exhibit similar magnitude, indicating that the classifier relies on a combination of many moderately important features rather than a single dominant component. It is also worth mentioning the fact that the important indices are spread across the entire EfficientNet feature vector, suggesting that discriminative information is distributed throughout the embedding space. While individual dimensions are not directly interpretable, this analysis confirms that the linear decision function makes systematic use of specific subsets of the representation.

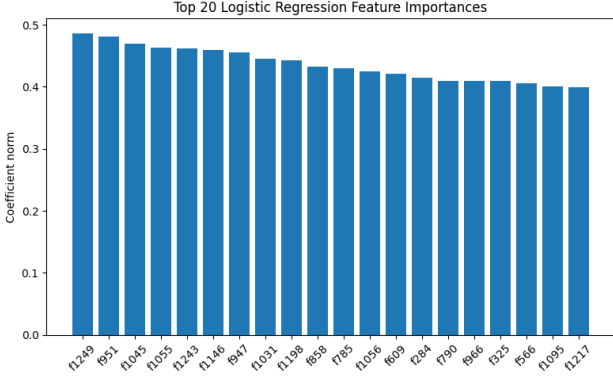


Figure 8. Feature Importance: Logistic Regression

Importantly, we should notice that feature importance in this context does not provide semantic interpretability: EfficientNet embedding dimensions do not translate to human-readable attributes, and their activation patterns emerge from high-level convolutional operations learned during pretraining. Consequently, the coefficients indicate which dimensions the linear classifier relies on the most, but **do not reveal which visual characteristics these dimensions represent**. Therefore, this analysis should be interpreted as an examination of how the classifier distributes weights across the embedding space, rather than an explanation of which specific visual factors influence the prediction.

4.6. Computational Cost and Training Time

To complement the accuracy-based evaluation, we can measure the computational footprint of the two main classical models used in this study: Logistic Regression (baseline) and the tuned SVM with RBF kernel (best-performing model). All experiments were conducted on Google Colab using an Intel Xeon CPU (2.20 GHz) and 12 GB RAM, with EfficientNet feature extraction executed on a Tesla T4 GPU.

To obtain reliable performance estimates, each timing experiment (training and inference) was repeated five times under identical conditions. It is reported the mean and standard deviation to account for variability due to CPU scheduling and transient system load. Model sizes were measured from the serialized `joblib` outputs.

Logistic Regression required approximately 1.74 s for training, with a very fast inference time of 3.3 ms for the entire test set (7 μ s per sample) and a model size of 41 KB. In contrast, the tuned SVM model required 12.66 s for training, 0.70 s for inference on 490 samples (1.43 ms per sample), and occupied 19 MB on disk. These results reflect the expected trade-off between the higher accuracy of the SVM and its substantially larger computational cost.

Table 3. Computational cost for Logistic Regression and tuned SVM.

Model	Training Time (s)	Inference Time (s)	Model Size
Logistic Regression	1.7441 $\pm$ 0.1610	0.0033 $\pm$ 0.0017	41 KB
SVM (RBF, tuned)	12.6600 $\pm$ 0.1576	0.6985 $\pm$ 0.0164	19 MB

5. Conclusions

In this work, we investigated how the choice of data, feature representation, and learning algorithms affect performance in a multi-class brain tumor classification task. By combining EfficientNet embeddings with classical machine learning models, we showed that high-quality, pretrained representations can enable simple classifiers such as Logistic Regression and SVM to achieve strong and meaningful results, even when training data is not abundant. Our results highlight the importance of careful evaluation, including cross-validation, learning curves and ROC analysis, to understand both the model capacity and its generalization power.

Although this study is not intended as a new state-of-the-art system, it demonstrates how modern representation learning can significantly enhance traditional methods and provide a solid baseline for future extensions.

Further work could explore end-to-end fine-tuning, more advanced interpretability techniques, or broader datasets to assess robustness and clinical applicability.

Code Availability

The full implementation used in this work is available at: <https://github.com/prollyyes/MLBrainTumorClassification.git>